GUION DE LA PRESENTACIÓN

Control semafórico multiagente con RND-MAPPO

Profesor: Jorge Alfredo López Jiménez · 17 diapositivas · unos 18 minutos · 4 expositores

Expositor	Diapositivas	Tema	Tiempo
1 · Tomás Castro	1 – 4	Portada, agenda, escenario y formulación	3:00
2 · Andrés Felipe Barbosa	5 – 7	Arquitectura, recompensa y configuración	3:15
3 · Santiago Sánchez	8 – 12	Resultados y demostración	7:00
4 · Mariana Vanoy	13 – 17	Análisis, dashboard, conclusiones y preguntas	4:30

Cifras para memorizar

Escenario: **4** intersecciones · **3600 s** · Δt = **5 s**

Actor: o_i → **128** → **64** → $|A_i|$ · Crítico: s → **256** → **128** → 1

PPO: ε = **0,2** · c_H = **0,01** · γ = **0,99** · λ_{GAE} = **0,95**

Entrenamiento: **100** episodios · **3** corridas

Aprendizaje: **12,0 → 3,8 s** · alcanza el fijo en el ep. **≈35**

Espera **5,73 → 2,91 s (−49 %)** · detenidos **−52 %**

Tiempo perdido **−35 %** · viaje **−14 %** · CO_2 **−12 %**

Cambios de fase **916 → 489** · todo con **p < 0,01**

Demanda ±20 %: espera **−42 a −51 %**

C2 / C3: espera **−51 % / −66 %** · C4: verde medio **3120 s**

Greedy: viaje **110 s** pero espera **165 s** (×29)

Evaluación: **5** semillas pareadas · **3** demandas · **26** métricas

Dos hallazgos del código que deben saber explicar:

1. sumo-rl 1.4.5 guarda *max_green* pero no lo aplica: solo impone el verde mínimo (15 s) y el amarillo (3 s). Por eso C4 sostiene una fase toda la hora.

2. Los pesos de la recompensa (w1, w2, w3, P) que recibe *train.py* no llegan a la recompensa del entorno, que usa los valores por defecto. La corrida «penaliza cambio» se entrenó con P = 0,5, no con 2,0.

EXPOSITOR 1 · TOMÁS CASTRO

1 · Control semafórico multiagente con RND-MAPPO

0:30 · acumulado 0:30

Buenos días, profesor. Somos Tomás Castro, Andrés Felipe Barbosa, Mariana Vanoy y Santiago Sánchez. Vamos a presentar un sistema de control semafórico basado en aprendizaje por refuerzo multiagente. Cada uno de los cuatro semáforos de un sector real de Bogotá es un agente que aprende, a partir de la simulación, cuándo cambiar de fase. Usamos el algoritmo MAPPO con un término de exploración llamado RND, entrenamos en el simulador de tráfico SUMO y comparamos contra los programas de tiempo fijo actuales.

2 · Agenda de la presentación

0:30 · acumulado 1:00

La presentación tiene cuatro partes. Primero, cómo formulamos el problema: el escenario y qué observa, qué decide y qué recompensa recibe cada semáforo. Segundo, la arquitectura del algoritmo, RND-MAPPO, y cómo lo entrenamos. Tercero, los resultados: aprendizaje, comparación con tiempo fijo, sensibilidad a la demanda y una demostración en vivo. Y cuarto, el análisis de la política aprendida, las variantes que probamos y las conclusiones.

3 · Descripción del escenario

1:00 · acumulado 2:00

El escenario es un sector real de Bogotá: tomamos la red vial de OpenStreetMap y la llevamos a SUMO, un simulador microscópico de tráfico, es decir, que simula cada vehículo con su origen, destino y modelo de conducción. En ese sector hay cuatro intersecciones semaforizadas, que llamamos C1 a C4.

Cada episodio es una hora simulada. Cada 5 segundos cada semáforo toma una decisión, así que un episodio son 720 pasos. Con la demanda nominal se completan unos 1.380 viajes por hora.

El entorno impone un verde mínimo de 15 segundos y 3 segundos de amarillo en cada cambio; el agente no puede saltárselos. Configuramos también un verde máximo de 60 segundos, pero al revisar el código de sumo-rl 1.4.5 encontramos que ese parámetro se guarda y no se aplica: el agente puede sostener una fase indefinidamente. Lo retomamos en los resultados por intersección. Desactivamos el teletransporte de SUMO para que ningún vehículo atascado desaparezca de las métricas. La referencia de comparación son los programas de tiempo fijo que trae la red.

4 · Formulación del problema de control

1:00 · acumulado 3:00

Formalmente, el problema es un juego estocástico parcialmente observable: hay cuatro agentes, uno por intersección, y ninguno ve el estado completo de la red.

Cada agente observa su propia intersección: qué fase tiene activa, si ya cumplió el verde mínimo, y la densidad y la cola de cada carril de entrada. La acción es elegir directamente cuál es la siguiente fase verde; si elige la misma, el verde continúa. La recompensa, que detallamos más adelante, premia que bajen la cola y la espera, y penaliza los cambios de fase.

Las intersecciones tienen distinto número de carriles y de fases, así que las observaciones y las acciones tienen dimensiones distintas: por eso cada agente tiene su propia red y no comparten pesos. El estado global, que solo usa el crítico, es la concatenación de las cuatro observaciones. El horizonte es de 720 pasos, con factor de descuento 0,99.

EXPOSITOR 2 · ANDRÉS FELIPE BARBOSA

5 · MAPPO: entrenamiento centralizado, ejecución descentralizada

1:15 · acumulado 4:15

El algoritmo es MAPPO, la versión multiagente de PPO, con el esquema de entrenamiento centralizado y ejecución descentralizada.

Cada intersección tiene un actor: una red neuronal que recibe la observación local y devuelve una distribución de probabilidad sobre las fases; la acción se muestrea de esa distribución. Tiene dos capas ocultas de 128 y 64 neuronas. Como cada intersección tiene distinto número de carriles y de fases, usamos cuatro actores independientes.

El crítico es una sola red, compartida, que recibe el estado global, es decir, las cuatro observaciones concatenadas, y estima el valor del estado. El crítico solo se usa para entrenar: le dice a cada actor qué tan buena fue su decisión para la red completa. En operación cada semáforo decide con su información local, sin comunicarse con los demás.

La actualización es la de PPO: una función objetivo recortada con épsilon 0,2 para que la política no cambie demasiado en cada paso, más un bono de entropía para mantener exploración. La ventaja se estima con GAE, con gamma 0,99 y lambda 0,95.

6 · Recompensa extrínseca y exploración por RND

1:15 · acumulado 5:30

La recompensa tiene dos términos. El extrínseco es el objetivo de control: en cada paso medimos cuánto cambió la cola y cuánto cambió la espera acumulada en la intersección. Si bajan, la recompensa es positiva; si suben, negativa. Es una recompensa diferencial, que es más estable que usar valores absolutos. Además restamos una penalización P de 0,5 cada vez que cambia la fase, para evitar oscilaciones. El término de peatones, w3, está en la formulación, pero no tiene efecto porque el escenario no tiene peatones cargados.

El término intrínseco es RND, Random Network Distillation. Hay una red g con pesos aleatorios congelados y una red predictora f que se entrena para imitarla. En estados que el agente ya visitó mucho, f predice bien y el error es bajo; en estados nuevos el error es alto, y ese error es el bono de exploración.

El peso λ de ese bono decae linealmente de 1 a 0,01 a lo largo del entrenamiento: al inicio el agente explora, y al final optimiza solo el tráfico.

7 · Configuración experimental

0:45 · acumulado 6:15

Estos son los hiperparámetros. Entrenamos 100 episodios, cada uno de una hora simulada. Las tasas de aprendizaje son 3 por 10 a la menos 4 para los actores y 10 a la menos 3 para el crítico; el recorte de PPO es 0,2 y el coeficiente de entropía 0,01. Al final de cada episodio se hacen 4 épocas de actualización. Repetimos el entrenamiento tres veces desde cero para medir la variabilidad.

Para evaluar, comparamos siempre de forma pareada: el agente y el tiempo fijo corren con la misma semilla de SUMO, o sea, exactamente el mismo tráfico. Usamos 5 semillas y tres niveles de demanda: 80, 100 y 120 % del nominal. Medimos 26 métricas: las del sistema que entrega sumo-rl, las de cada viaje a partir del archivo tripinfo de SUMO (tiempo de viaje, tiempo perdido, paradas), las emisiones y el comportamiento del control. La significancia la evaluamos con prueba t pareada.

EXPOSITOR 3 · SANTIAGO SÁNCHEZ

8 · Curvas de aprendizaje

1:00 · acumulado 7:15

Esta es la curva de aprendizaje. Cada punto es un episodio de entrenamiento de una hora; el eje vertical es la espera media por vehículo, suavizada con media móvil de 5 episodios.

En los primeros episodios la política es prácticamente aleatoria y la espera está entre 11 y 16 segundos, por encima del tiempo fijo, que es la línea punteada de 5,7 segundos. Alrededor del episodio 35 el agente alcanza el nivel del tiempo fijo y hacia el episodio 60 se estabiliza. En la corrida base la espera pasa de 12 segundos en los primeros 10 episodios a 3,8 en los últimos 10.

Las tres corridas independientes convergen al mismo nivel, entre 3,8 y 4 segundos: el aprendizaje es reproducible.

Una aclaración: la línea del tiempo fijo viene de la evaluación, porque el entrenamiento solo corre el agente.

9 · Desempeño frente al control de tiempo fijo

1:30 · acumulado 8:45

Estos son los resultados de la política base frente al tiempo fijo, con la demanda nominal y 5 semillas pareadas.

La espera media baja 49 %, de 5,7 a 2,9 segundos, y el número promedio de vehículos detenidos baja 52 %. Las métricas por viaje, que salen de tripinfo, confirman la mejora: el tiempo perdido por viaje, es decir, el tiempo adicional frente a circular a velocidad libre, baja 35 %, y el tiempo de viaje total 14 %. También baja 39 % el percentil 95 de la espera por viaje, o sea que la mejora no solo es en promedio sino también para los vehículos más afectados.

Como hay menos detenciones, la velocidad media sube 17 % y el CO_2 baja 12 %. El throughput sube ligeramente, 0,8 %. Y el agente cambia de fase 47 % menos veces que el programa fijo.

Todas las diferencias son significativas con prueba t pareada; los p-valores están por debajo de 0,001, salvo el de viajes completados, que es 0,001.

10 · Sensibilidad al nivel de demanda

0:45 · acumulado 9:30

Para medir la robustez, evaluamos la misma política, entrenada solo con la demanda nominal, con 20 % menos y 20 % más vehículos, sin reentrenar.

En los tres niveles el agente mejora con significancia: la espera media baja entre 42 y 51 %, y el tiempo perdido por viaje entre 33 y 39 %. El pico de vehículos detenidos también baja: con 120 % de demanda pasa de 46 a 29.

La ventaja se reduce un poco al aumentar la demanda, que es lo esperable porque la red se acerca a saturación, pero se mantiene. Eso indica que la política generaliza a condiciones distintas de las de entrenamiento.

11 · Desempeño por intersección

0:45 · acumulado 10:15

Desagregando por intersección: la congestión está en C2 y C3. Ahí la espera acumulada en los carriles de entrada baja 51 y 66 %. El patrón de control cambia: el tiempo fijo cambia de fase cada 10 a 14 segundos en promedio, y el agente sostiene el verde unos 20 segundos, con la mitad de cambios. Es decir, aprende a servir más tiempo la dirección con cola, en lugar de alternar por reloj.

C4 es un caso particular: el agente prácticamente no cambia de fase en toda la hora, con un verde medio de 3.120 segundos, y aun así la espera ahí es cero. Eso indica que la vía transversal de C4 casi no tiene demanda en este escenario. Hay que tener cuidado con esto: como mencionamos, sumo-rl no aplica el verde máximo, y en un semáforo real esta política sería inaceptable si llega un vehículo por la transversal. Lo dejamos como trabajo futuro.

12 · Ejecución en tiempo real en sumo-gui

3:00 · acumulado 13:15

DEMO. Cambiar a la terminal de VS Code en la carpeta del proyecto y correr:

.\.venv\Scripts\python.exe demo_en_vivo.py --minutos 10 --delay 300 --escala 1.2

Mientras abre: "Vamos a correr dos simulaciones en paralelo con exactamente la misma demanda y la misma semilla. La de la izquierda usa los programas de tiempo fijo y la de la derecha la política aprendida. Usamos 20 % más demanda que la nominal para que el efecto sea visible."

Mientras corre: señalar las colas en C2 y C3. En el tiempo fijo, el verde alterna por reloj aunque una de las aproximaciones esté vacía; con el agente, el verde se sostiene en la aproximación con cola.

Leer cada línea del marcador: detenidos y espera, fijo frente a agente.

Al final, leer la línea de resumen. Como referencia, en la prueba de 15 minutos con demanda nominal obtuvimos 48 % menos detenidos y 57 % menos espera.

Plan B: si falla, poner el video grabado.

EXPOSITOR 4 · MARIANA VANOY

13 · Política estocástica frente a política greedy

1:15 · acumulado 14:30

Un resultado que vale la pena analizar. El actor entrega una distribución de probabilidad sobre las fases. En la evaluación muestreamos de esa distribución, igual que en el entrenamiento. La alternativa es la política greedy, que siempre elige la fase más probable.

Si miramos solo las métricas por viaje, la greedy parece la mejor: el tiempo de viaje baja a 110 segundos y el tiempo perdido a 28. Pero la espera media del sistema sube a 165 segundos, 29 veces la del tiempo fijo.

La explicación es un sesgo de supervivencia. Las métricas por viaje solo cuentan los vehículos que llegan a su destino. La política greedy concentra el verde en pocas fases y deja otras aproximaciones casi sin servicio; esos vehículos se acumulan, no terminan el viaje y quedan fuera del promedio. Por eso hay más viajes sin terminar y menos completados.

Dos conclusiones: con 100 episodios la política todavía no es lo bastante determinista para usarse en modo greedy, y la evaluación debe combinar métricas de sistema y por viaje.

14 · Variantes de entrenamiento

1:00 · acumulado 15:30

Además de la configuración base, entrenamos cuatro variantes. Las dos primeras repiten la configuración con otra semilla de entrenamiento. Las dos reducen el tiempo de viaje con significancia, 12 y 13 %, pero su espera media es más variable: con algunas semillas de evaluación se acercan al tiempo fijo, y por eso la reducción de espera no es significativa en esas corridas. La corrida base es la más estable.

Con tasas de aprendizaje tres veces menores y solo 50 episodios, el agente no alcanza a converger y queda 42 % peor que el tiempo fijo.

Sobre la variante de penalización: la intención era subir P de 0,5 a 2,0. Al revisar el código encontramos que el parámetro se pasa al calculador de la recompensa total, pero la recompensa extrínseca se calcula dentro del entorno con los valores por defecto. Esa corrida se entrenó en la práctica con P igual a 0,5, es decir, equivale a la base con 50 episodios. Hay que corregir el código y repetirla antes de sacar conclusiones sobre la penalización.

15 · Dashboard de seguimiento del modelo

1:00 · acumulado 16:30

Para hacer seguimiento al modelo estamos construyendo un dashboard. Esta imagen es el prototipo de la interfaz; los números que aparecen son de ejemplo, no son nuestros resultados.

La idea es tener tres vistas: evaluación, entrenamiento y simulación. En la vista de evaluación se ven las métricas de tráfico, como tiempo de espera, longitud de cola y tiempo de parada, y el estado de cada uno de los cuatro agentes. Abajo, la curva de recompensa durante el entrenamiento.

Los datos ya están listos: el script exportar_datos.py consolida todos los resultados en siete archivos CSV en formato largo, con un diccionario de datos: las curvas de entrenamiento por episodio, cada episodio de evaluación por semilla y nivel de demanda, el resumen frente al tiempo fijo con intervalos de confianza y p-valores, los resultados por intersección y las series de tiempo dentro de la hora.

El siguiente paso es conectar la interfaz a esos archivos y a los registros de entrenamiento que ahora genera train.py, como las pérdidas, la entropía y la recompensa intrínseca.

16 · Conclusiones

1:00 · acumulado 17:30

Para cerrar, cuatro conclusiones.

Primera: con entrenamiento centralizado y ejecución descentralizada, los cuatro controladores aprenden una política de fase en unos 60 episodios, y el resultado se reproduce en tres corridas independientes.

Segunda: frente al tiempo fijo, la espera baja 49 %, el tiempo perdido por viaje 35 % y el CO_2 12 %, con la mitad de cambios de fase. Todo significativo.

Tercera: la mejora se mantiene con 20 % más o menos demanda sin reentrenar.

Cuarta: la evaluación necesita varias métricas; las métricas por viaje, solas, pueden ocultar vehículos retenidos, como vimos con la política greedy.

Como trabajo futuro: aplicar el verde máximo en el entorno, que hoy sumo-rl no impone, y corregir el paso de los pesos de la recompensa para repetir la variante con P igual a 2; simular la hora pico, que en el escenario empieza a los 5.000 segundos y queda fuera de nuestro episodio de 3.600; estabilizar el entrenamiento entre semillas, y agregar peatones.

17 · Gracias. ¿Preguntas?

0:15 · acumulado 17:45

Muchas gracias. Quedamos atentos a sus preguntas.

Preguntas probables y respuesta corta:

¿Por qué MAPPO y no un solo agente para toda la red? Un agente central tendría un espacio de acciones combinatorio (el producto de las fases de las cuatro intersecciones) y requeriría comunicación en operación. Con CTDE cada controlador decide localmente y el crítico central solo se usa al entrenar.

¿Por qué RND? Porque la recompensa diferencial es poco informativa al inicio; el bono de exploración evita converger temprano a una política subóptima. λ decae a 0 para que al final optimice solo el tráfico.

¿Para qué sirve el dashboard? Para seguir entrenamiento y evaluación sin abrir los CSV: lee los archivos que genera exportar_datos.py. Hoy es un prototipo con valores ilustrativos.

¿Por qué muestrear y no usar argmax? Ver la diapositiva 13: con 100 episodios la distribución no es lo bastante concentrada y el argmax deja aproximaciones sin servicio.

¿Qué tan significativo es con 5 semillas? Usamos prueba t pareada: el agente y el tiempo fijo ven el mismo tráfico, lo que reduce la varianza; los p-valores están por debajo de 0,001.

¿Qué varía entre semillas de evaluación? La demanda está definida por archivos de rutas fijos; la semilla cambia el comportamiento estocástico de los conductores en SUMO. Por eso también evaluamos con tres niveles de demanda.

¿Se podría implementar en campo? Faltaría aplicar el verde máximo, evaluar la hora pico y los peatones, y validar con conteos reales. Cada controlador solo necesita sensores de su propia intersección.

¿Cuánto tarda entrenar? Del orden de medio minuto de cómputo por hora simulada; 100 episodios, alrededor de una hora en CPU.